

Diseño Físico FST

Traducción del esquema lógico definitivo a un esquema implementable en un gestor concreto. Cubre las seis actividades del método, y las seis ya tienen sus decisiones tomadas. Lo que queda de aquí en adelante es ejecutarlas, no decidir las.

ENTRADA	PROCESO	SALIDA
Esquema lógico definitivo · 16 relaciones en BCNF	Traducción a las estructuras de PostgreSQL	Esquema físico · decidido, falta ejecutar
ESTADO		
Decisiones cerradas · 6 de 6 actividades		

01	Selección del gestor de base de datos	RESUELTA
02	Representación del esquema lógico en el gestor	RESUELTA
03	Organización de archivos e índices	RESUELTA
04	Seguridad y control de acceso	RESUELTA
05	Rendimiento y redundancia controlada	RESUELTA
06	Documentación de la implementación	RESUELTA

EN QUÉ SENTIDO ESTO ESTÁ TERMINADO, Y EN CUÁL NO

Los documentos de diseño conceptual y lógico describen etapas **cerradas** con resultado validado. Este documento llegó a un punto parecido pero no idéntico: **las seis actividades tienen ya su decisión tomada**, con fecha y responsable en cada una. Lo que falta no es decidir — es escribir el código: el DDL completo de las 16 relaciones, el disparador de D-04, el cron del respaldo de D-18.

En verde, lo que decidió el equipo en sesión de trabajo, con registro en `preguntas-doctor.md`. En ámbar, lo que este documento propone como forma concreta de ejecutar esas decisiones —los tipos exactos de PostgreSQL, la convención de nombres de las restricciones— y que nadie ha revisado línea por línea todavía.

ACTIVIDAD 1 RESUELTA

Selección del gestor de base de datos

«Seleccionar el Sistema Gestor de Base de Datos en el cuál se implementará la base de datos e identificar sus funcionalidades.»

METODOLOGÍA DE DISEÑO · ETAPA FÍSICA · ACTIVIDAD 1

LOS EJEMPLOS DEL LIBRO

El apéndice A implementa su caso en PostgreSQL; el apéndice B implementa otro caso distinto en Oracle.

Son dos ejemplos paralelos, no una recomendación: el libro **no argumenta ninguna de las dos elecciones**, igual que no argumentó el modelo relacional en la etapa lógica.

ESTE PROYECTO · FST

PostgreSQL, decidido por el equipo el 13 de septiembre de 2026 y registrado como **D-11**.

El capítulo 3 del TT ya lo mencionaba, pero eso no bastaba: el documento escrito es la fuente de **menor** autoridad del proyecto. Lo que cierra la actividad es la decisión, no la mención.

Por qué PostgreSQL y no MySQL

La comparación se hizo contra las necesidades reales del sistema, sin apoyarse en lo que el libro usa ni en lo que el capítulo 3 ya decía. Dos criterios decidieron.

CRITERIO	QUÉ EXIGE ESTE SISTEMA	POR QUÉ INCLINA LA BALANZA
Consultas de los reportes	OBSERVACION → INTERVALO → PRESENTA es una medición por minuto y por espécimen. Los reportes comparan los tres grupos entre sí, que es lo que el laboratorio declara como objetivo del experimento	Son consultas de agregación y comparación entre subconjuntos. PostgreSQL tiene mejor soporte de funciones de ventana y de agregación que MySQL, y esas consultas se van a escribir muchas veces
Reproducibilidad	CONFIGURACION guarda los umbrales con los que corrió cada análisis, para poder comparar corridas en TT-II. Ocho de sus columnas forman una clave alterna: si los parámetros coinciden, debe ser la misma fila	Eso solo funciona si el gestor rechaza lo que no cuadra en vez de acomodarlo. PostgreSQL es estricto por omisión; MySQL, salvo que se active el modo estricto, trunca en lugar de rechazar
Costo y licencia	Proyecto escolar, sin presupuesto	No decide: los dos son libres y gratuitos
Volumen y concurrencia	Un laboratorio, pocos investigadores a la vez	No decide: cualquiera de los dos sobra para esta escala

QUÉ SE ACEPTA A CAMBIO

Nada relevante para este proyecto. La ventaja histórica de MySQL —presencia en hospedaje compartido barato— no aplica: el sistema corre con un módulo de visión por computadora y un clasificador, así que va a vivir en un servidor propio del laboratorio o en una máquina dedicada, no en hospedaje compartido.

ACTIVIDAD 2 RESUELTA

Representación del esquema lógico en las estructuras del gestor

«Representar e implementar los elementos del Esquema Lógico en las estructuras permitidas por el Modelo Físico que sigue el Sistema Gestor, teniendo en cuenta todas las consideraciones estructurales y no estructurales.»

METODOLOGÍA DE DISEÑO · ETAPA FÍSICA · ACTIVIDAD 2

Lo que entra a esta actividad ya está completo y validado: **16 relaciones en BCNF, 73 atributos y 17 llaves foráneas**, todas de una sola columna. Lo que falta es traducir cada

elemento a un tipo, una restricción o una estructura que PostgreSQL entienda.

RELACIONES 16 11 del laboratorio + 4 de soporte del sistema + Administrador	ATRIBUTOS 73 incluidas las llaves foráneas	LLAVES FORÁNEAS 17 todas de una sola columna	DOMINIOS POR FIJAR 0 los ocho, resueltos
--	---	---	---

Parte 1 — De dominio lógico a tipo de PostgreSQL

El esquema lógico describe los dominios en lenguaje natural: «identificador», «texto», «entero ≥ 1 », «real». Esta actividad les asigna un tipo concreto. La tabla siguiente es la regla que se aplicó; todas las filas son **propuesta de este documento**.

DOMINIO LÓGICO	TIPO PROPUESTO	POR QUÉ ESE Y NO OTRO
Identificador subrogado	<code>integer generated always as identity</code>	Es la forma del estándar SQL y sustituye a <code>serial</code> , que PostgreSQL conserva solo por compatibilidad. «Always» impide que la aplicación inserte el identificador a mano
Identificador institucional USUARIO	<code>varchar(10)</code>	No es numérico aunque lo parezca: puede llevar ceros a la izquierda. Por D-02 el mismo campo acepta boleta de estudiante o número de empleado; 10 caracteres cubre el formato de boleta del IPN. Decidido por el equipo el 13-sep
Texto corto	<code>varchar(n)</code>	Nombres, etiquetas, correos. El apéndice del libro usa <code>character(n)</code> de longitud fija, que rellena con espacios y ensucia las comparaciones — aquí se descarta a propósito
Texto libre y rutas	<code>text</code>	<code>notas</code> , <code>mensaje</code> , <code>archivo</code> , <code>ruta</code> . No hay una longitud defendible que imponer, y en PostgreSQL <code>text</code> no cuesta más que <code>varchar</code>
Fecha sin hora	<code>date</code>	Solo <code>EXPERIMENTO.fecha</code> : es el día del experimento, no un instante

Marca de tiempo	timestampz	Las cuatro fechas de registro. Con zona horaria, para que la hora de carga de un video no dependa de la configuración de la máquina que lo insertó
Umbrales y confianza	numeric(p,s)	No real ni double precision . La clave alterna de CONFIGURACION compara ocho umbrales por igualdad, y la igualdad entre flotantes no es fiable. Con numeric el valor almacenado es el valor escrito
Segundos y duración	numeric(p,s)	PRESENTA.segundos y VIDEO.duracion . Mismo argumento: se suman y se comparan contra el minuto y contra la sesión declarada
Contadores pequeños	smallint	minuto , ordinal , nCilindros , numeroRata , numeroCilindro . Ninguno pasa de dos dígitos
Booleano	boolean	USUARIO.activo (D-10) y NOTIFICACION.leido
Hash del modelo	char(64)	Es el único caso donde la longitud fija sí corresponde, porque un hash siempre mide lo mismo. Se calcula con SHA-256 (hashlib.sha256(archivo).hexdigest() de Python, sin implementar nada propio) — decidido por el equipo el 13-sep. 64 es la longitud fija de lo que produce SHA-256. No confundir con el hash de contraseñas, que usa bcrypt (ver actividad 4)

Parte 2 — Los dominios por extensión

La etapa lógica cerró con **seis** dominios por extensión, y el libro muestra exactamente cómo se declaran: `CREATE DOMAIN ... CHECK (VALUE IN (...))`. De esos seis, **cinco ya se pueden escribir**; uno es en realidad un rango numérico, no una lista, y se maneja como `CHECK` en la Parte 1. **Aparte, y sin haber estado en esa cuenta de seis**, había dos dominios más de `ANALISIS` que surgieron después — los dos ya están resueltos (D-16, D-17).

```

-- los cinco que ya tienen extensión documentada
CREATE DOMAIN tipo_grupo AS varchar(24)
    CHECK (VALUE IN ('control', 'referencia', 'tratamiento experimental'));

CREATE DOMAIN sesion_video AS varchar(6)
    CHECK (VALUE IN ('Día 1', 'Día 2'));

CREATE DOMAIN formato_reporte AS varchar(4)
    CHECK (VALUE IN ('CSV', 'XLSX', 'PDF'));

CREATE DOMAIN tipo_notificacion AS varchar(32)
    CHECK (VALUE IN ('análisis completado', 'error del pipeline', 'alerta de disco'));

CREATE DOMAIN estado_analisis AS varchar(10)
    CHECK (VALUE IN ('en cola', 'procesando', 'completado', 'error'));
    -- D-15, 13-sep: son 4, no 3. Falta "en cola" para el momento entre que se
    -- pide el análisis y que el Worker lo toma de la cola (cap. 1 y 3 del TT
    -- ya piden esa cola; el cap. 5, para el modelo viejo, ya traía el cuarto
    -- valor y este documento no lo había cruzado con el esquema)

-- el sexto no es una lista, es un rango: CONFIGURACION.umbralConfianza
-- se declara CHECK (valor BETWEEN 0 AND 1) sobre la columna, no como CREATE DOMAIN

-- estos dos no estaban en la cuenta de seis; surgieron después
CREATE DOMAIN etapa_analisis AS varchar(16)
    CHECK (VALUE IN ('preprocesamiento', 'deteccion', 'seguimiento', 'clasificacion'));
    -- D-16, 13-sep: el cap. 4 (RF) ya las nombraba como requisito, y el cap. 5
    -- las repite como Módulo 1-4. Se acortaron los calificativos ("de
    -- cilindros", "de conducta") para que quede tan corto como los demás

CREATE DOMAIN nivel_clasif_analisis AS varchar(8)
    CHECK (VALUE IN ('preciso', 'agrupado'));
    -- D-17, 13-sep: RN-13 (cap. 4) obliga a que el reporte indique si el
    -- clasificador distinguió las tres conductas por separado (preciso) o
    -- tuvo que agrupar nado activo y escalamiento en "conducta activa"
    -- (agrupado), por no alcanzar F1 >= 85%. El cap. 4 nunca nombró los dos
    -- lados de la regla – los nombres son de esta decisión, no del .tex

```

CON ESTO, LOS SEIS DOMINIOS DE LA ETAPA LÓGICA MÁS LOS DOS QUE SURGIERON DESPUÉS QUEDAN RESUELTOS

Nada de esta actividad depende ya de una decisión sin tomar. Lo que queda es ejecutar: escribir el DDL completo de las 16 relaciones con todos los tipos y dominios ya fijados.

Parte 3 — **CONDUCTA** es la excepción deliberada

Las conductas parecen el candidato más obvio a dominio cerrado, y sin embargo se quedan como tabla. La razón viene de la etapa lógica y sigue vigente: su dominio tiene **dos extensiones según el modo del clasificador** —{nado activo, inmovilidad, escalamiento} en modo completo, {conducta activa, inmovilidad} en modo degradado—, y un dominio cerrado tendría que listar los cuatro valores sin nada que impida mezclar unos con otros en el mismo análisis.

Como tabla, y junto con `ANALISIS.nivelClasif`, sí se puede expresar qué extensión aplicó a cada análisis. Es la única relación del esquema que existe por una razón de esta clase.

Parte 4 — La forma de las tablas

Un ejemplo completo, con todos los tipos de restricción que el esquema necesita. Se eligió `ESPECIMEN` porque concentra los cuatro casos: llave subrogada, foránea, clave alterna y restricción semántica.

```
CREATE TABLE ESPECIMEN
(
    idEspecimen      integer GENERATED ALWAYS AS IDENTITY,
    idTanda           integer NOT NULL,
    numeroRata        smallint NOT NULL,
    numeroCilindro    smallint NOT NULL,

    CONSTRAINT PK_ESPECIMEN PRIMARY KEY (idEspecimen),

    CONSTRAINT AK_ESPECIMEN_CILINDRO UNIQUE (idTanda, numeroCilindro),

    CONSTRAINT FK_TANDA_ESPECIMEN FOREIGN KEY (idTanda)
        REFERENCES TANDA (idTanda) MATCH SIMPLE
        ON UPDATE NO ACTION ON DELETE NO ACTION,

    CONSTRAINT CK_ESPECIMEN_RATA CHECK (numeroRata >= 1)
```

```
-- falta: numeroCilindro <= nCilindros de su tanda, que mira otra tabla
-- la unicidad de numeroRata dentro del grupo se agrega en la actividad 5 (D-04) – ver
);
```

LA POLÍTICA DE LAS 17 LLAVES FORÁNEAS

Todas se declaran `ON UPDATE NO ACTION ON DELETE NO ACTION`, que es lo que hace el apéndice del libro en cada una de las suyas. La consecuencia es deliberada: **la base de datos rechaza el borrado en vez de propagarlo**. Nadie borra un experimento y se lleva por delante sus grupos, tandas, especímenes, observaciones e intervalos sin darse cuenta.

Es la misma lógica que sostiene `USUARIO.activo` (**D-10**): si borrar está prohibido de hecho, hace falta una forma de dar de baja que no sea borrar.

Parte 5 — Dónde este esquema se aparta del ejemplo del libro

EL APÉNDICE HACE	AQUÍ SE HACE	POR QUÉ
<code>character(n)</code>	<code>varchar(n)</code> · <code>text</code>	La longitud fija rellena con espacios a la derecha. En comparaciones y en exportación a CSV eso aparece como basura invisible
Llaves naturales	Llaves subrogadas	No es una desviación de esta etapa: se decidió en la lógica, porque la transformación literal producía llaves de hasta siete columnas sobre una sola de dato
<code>money</code>	No se usa	No hay ningún atributo monetario en este esquema
Restricciones sin nombre en algunos casos	Todas con nombre	<code>PK_</code> , <code>AK_</code> , <code>FK_</code> , <code>CK_</code> . Cuando una restricción falla en producción, el

nombre es lo único que dice qué regla se violó

ACTUALIZADO 13-SEP · LOS DOS HUECOS DE ESTA ACTIVIDAD YA SE CERRARON

`ADMINISTRADOR(idInstitucional* [FK])` ya está en `grafo_relacional_reconciliado.puml`, en relación (1,1)–(0,1) con `USUARIO` (**D-08**). Su llave primaria es la misma foránea, así que PostgreSQL la indexa sola al declarar el `PRIMARY KEY` — entra a la columna de índices gratuitos en la actividad 3, no a la de índices por declarar.

La discrepancia de nombre también se resolvió: el grafo relacional ya dice `contrasenaHash`, igual que el diagrama de clases. La tabla de tipos de la Parte 1 y el ejemplo de `ESPECIMEN` no la mencionan directamente, pero la regla de **texto corto** → **varchar(n)** de esa misma parte le aplica sin cambios.

ACTIVIDAD 3 RESUELTA

Organización de archivos e índices

«Determinar la organización de los archivos y los índices requeridos según el análisis de los datos que serán almacenados y las transacciones previstas, en esta actividad debe considerarse la estimación del espacio en disco.»

METODOLOGÍA DE DISEÑO · ETAPA FÍSICA · ACTIVIDAD 3

Esta actividad tiene una entrada que ya existe: las **once transacciones verificadas** en la actividad 6 de la etapa lógica. Ahí se comprobó que el esquema las soporta; aquí se mira qué índices necesitan para no recorrer tablas enteras.

El punto de partida es un detalle del gestor que decide casi todo el trabajo: **PostgreSQL crea el índice de las llaves primarias y de las restricciones de unicidad, pero no el de las llaves foráneas**. Una foránea sin índice propio obliga a recorrer la tabla hija completa cada vez que se navega desde el padre.

9 - YA QUEDAN INDEXADAS	8 - HAY QUE DECLARAR EL ÍNDICE
son la primera columna de una clave ya declarada GRUPO.idExperimento → EXPERIMENTO clave alterna (idExperimento, etiqueta) TANDA.idGrupo → GRUPO clave alterna (idGrupo, ordinal) ESPECIMEN.idTanda → TANDA clave alterna (idTanda, numeroCilindro) VIDEO.idTanda → TANDA clave alterna (idTanda, sesion) OBSERVACION.idEspecimen → ESPECIMEN clave alterna (idEspecimen, idVideo) INTERVALO.idObservacion → OBSERVACION clave alterna (idObservacion, minuto) PRESENTA.idIntervalo → INTERVALO primera columna de la llave primaria CONFIGURACION.hashModelo → MODELO si la clave alterna se declara con el hash al frente ADMINISTRADOR.idInstitucional → USUARIO es también su llave primaria - subtipo, D-08	ninguna clave existente las cubre EXPERIMENTO.idInstitucional → USUARIO experimentos de un investigador ANALISIS.idVideo → VIDEO análisis de un video - la ruta de todo resultado ANALISIS.idConfig → CONFIGURACION con qué umbrales corrió - reproducibilidad OBSERVACION.idVideo → VIDEO es la segunda columna de su clave alterna PRESENTA.conducta → CONDUCTA es la segunda columna de la llave primaria REPORTE.idExperimento → EXPERIMENTO reportes de un experimento NOTIFICACION.idInstitucional → USUARIO el panel del investigador, la consulta más frecuente NOTIFICACION.idExperimento → EXPERIMENTO admite nulo - el índice no indexa los nulos

Figura 1. Reparto de las diecisiete llaves foráneas — nueve no cuestan nada porque el índice ya viene con una clave alterna que empieza por esa columna, o porque la propia foránea es la primaria de un subtipo; las otras ocho hay que declararlas. Las dos de la derecha marcadas «segunda columna» son el caso menos evidente: la clave alterna existe, pero un índice compuesto no sirve para buscar por su segunda columna.

Cuántas filas hay que indexar

Antes de justificar índices conviene saber el tamaño real. Las multiplicidades del grafo relacional permiten estimarlo sin medir nada: un experimento mínimo son tres grupos, dos tandas por grupo, tres o cuatro especímenes por tanda y una o dos sesiones grabadas.

RELACIÓN	FILAS POR EXPERIMENTO	DE DÓNDE SALE
GRUPO	3	Control, referencia y tratamiento experimental
TANDA	6	Dos tandas por grupo
ESPECIMEN	~24	Tres o cuatro por tanda
VIDEO	~12	Una o dos sesiones por tanda
OBSERVACION	~48	Cada espécimen aparece en una o dos sesiones
INTERVALO	~240	Cinco minutos por observación

RELACIÓN	FILAS POR EXPERIMENTO	DE DÓNDE SALE
PRESENTA	~500 a 700	Dos o tres conductas por intervalo

EL ESPACIO EN DISCO NO ES PROBLEMA DE LA BASE DE DATOS

Un experimento completo cabe en **menos de mil quinientas filas**, casi todas de tres o cuatro columnas cortas. Cien experimentos siguen siendo del orden de cien mil filas: para PostgreSQL eso es nada, y ningún índice de esta lista se justifica por volumen.

El disco lo consumen **los videos**, que no viven en la base de datos — `VIDEO.archivo` guarda la ruta, no el contenido. Por eso la estimación de espacio que esta actividad pide corresponde en realidad al almacenamiento de archivos, y por eso existe la regla de negocio que los borra a los treinta días de completado el análisis.

La consecuencia práctica: los ocho índices de la derecha se declaran **por corrección de las consultas, no por tamaño**. Y conviene decirlo así en la defensa, en vez de justificarlos con un argumento de rendimiento que a esta escala no se sostiene.

LA OTRA MITAD DE LA ACTIVIDAD — DÓNDE VIVEN LOS DATOS

Esto ya estaba resuelto, solo que en otros capítulos del TT y nunca cruzado con esta actividad. Los datos se reparten en **tres lugares**: PostgreSQL para todo lo estructurado, un volumen compartido para los videos que suben los investigadores, y un volumen aparte y dedicado para los modelos entrenados (`rat.pt` , `resnet18_fst.pt` , `resnet50_fst.pt`) que el Worker monta al arrancar (cap. 5).

La política de retención de los videos también está escrita: se borran automáticamente a los 30 días de terminado el análisis (**RNF-08**), mientras que los reportes y resultados se conservan indefinidamente aunque el video ya no exista (**RN-06**). Y hay un mecanismo de capacidad: alerta al Administrador al 80% de uso de disco, limpieza automática de los videos más antiguos al 90% (**RNF-09**).

D-18 · RESUELTO (EQUIPO, 13-SEP) · RESPALDO DE LA BASE DE DATOS

Esto sí era un hueco real: a diferencia de la retención de videos, **ningún .tex menciona con qué frecuencia o herramienta se respalda la base de datos** — son dos cosas

distintas, esta es sobre PostgreSQL completo (experimentos, resultados, cuentas), no sobre los archivos de video.

Respaldo automático diario con pg_dump , conservando los últimos 7 días. Suficiente para un laboratorio de este tamaño, sin herramientas adicionales — pg_dump viene incluido con PostgreSQL.

ACTIVIDAD 4 RESUELTA

Seguridad y control de acceso

«Implementar mecanismos de seguridad y control de acceso a los datos.»

METODOLOGÍA DE DISEÑO · ETAPA FÍSICA · ACTIVIDAD 4

Esta actividad tenía el hueco más grande del documento: una **contradicción entre dos cosas que el proyecto ya había dado por buenas** y que nunca se pusieron una junto a la otra. Ya se resolvió con el equipo (D-12, D-13).

LO QUE DECÍA LA ETAPA LÓGICA

La actividad 5 —integración de vistas— se había cerrado como «no aplica»: una sola vista, sin permisos diferenciados.

Y entre las transacciones verificadas había una que lo confirmaba: «consultar experimentos de cualquier investigador — sin restricción de acceso».

LO QUE DECÍA D-08

El administrador sí tiene permisos que un investigador no tiene: crear cuentas, desactivarlas, listar usuarios, monitorear el sistema.

Se modeló como subtipo, así que la diferencia de permisos ya estaba en el esquema — pero nadie había dicho **quién la hace cumplir**.

D-12 · RESUELTO (EQUIPO, 13-SEP)

Sí hay una vista diferenciada, pero es una sola aplicación con un solo inicio de sesión — no dos sistemas separados. A un Administrador le aparece una pestaña o sección adicional que a un Investigador no le aparece, y ahí ejecuta las cuatro acciones de D-08.

El control vive **en la aplicación**, no en roles de PostgreSQL, y es doble a propósito:

- **Se esconde:** esa sección no se muestra en la interfaz si el usuario no está en `ADMINISTRADOR`.
- **Se bloquea:** antes de ejecutar cualquiera de las cuatro acciones, el sistema verifica el rol —sin importar por dónde llegue la petición, no solo cuando viene del botón.

Esconder sin bloquear no basta: si alguien encuentra la forma de pedirle al sistema esa acción directamente —sin pasar por la pantalla—, y el sistema no vuelve a preguntar quién es, la ejecuta igual. Con las dos cosas juntas, ni se ve el botón ni se puede entrar por otro lado.

D-13 · RESUELTO (EQUIPO, 13-SEP)

La transacción «sin restricción de acceso» entre investigadores **sigue vigente**, y ahora con motivo explícito: necesitan ver los experimentos de los demás **para poder comparar sus resultados** entre sí. Es una restricción distinta e independiente de D-12 — esta es entre investigadores, D-12 es investigador contra administrador.

Qué falta ejecutar, ya con la decisión tomada

PIEZA

QUÉ FALTA

La comprobación de rol	Escribir la función que revisa si <code>idInstitucional</code> aparece en <code>ADMINISTRADOR</code> , y llamarla antes de las cuatro acciones y antes de mostrar la sección. Una sola función, usada en los dos lugares, para que no se desincronicen
La sesión	Con qué se identifica una sesión activa (token, cookie firmada) no se ha decidido — es lo que la comprobación de rol necesita para saber quién está pidiendo la acción

Y POR DEBAJO, UNA PREGUNTA ANTERIOR QUE SIGUE ABIERTA

Q-08 pregunta si existe autoregistro con aprobación o si solo el administrador crea cuentas. **D-01** ya la respondió del lado del equipo —solo el administrador crea cuentas— y dejó dicho que el capítulo 1 del TT está mal escrito y hay que corregirlo. Mientras esa corrección no se aplique, el documento y el sistema dicen cosas distintas sobre quién puede entrar.

Fuera de eso, las contraseñas se almacenan como hash, nunca en claro, con `bcrypt` y `sal` — ya documentado en el cap. 4 (RNF-05) y el cap. 5, es lo único de esta actividad que el propio LaTeX resuelve sin necesitar una decisión nueva.

Rendimiento y redundancia controlada

«Garantizar el rendimiento de la base de datos y tomar acciones como incluir **redundancia controlada** en caso de ser necesario buscando mejorar el desempeño del sistema.»

METODOLOGÍA DE DISEÑO · ETAPA FÍSICA · ACTIVIDAD 5

La etapa lógica llegó aquí con una candidata ya identificada y con **dos motivos** para ella, uno de los cuales no es de rendimiento en absoluto. Ya se decidió con el equipo (D-04).

D-04 · RESUELTO (EQUIPO, 13-SEP) · REDUNDANCIA CONTROLADA

La 3FN obligó a quitar `ESPECIMEN.idGrupo` : el grupo de un espécimen se alcanza por su tanda, así que guardarlo era una dependencia transitiva. Correcto — pero dejó sin forma de declarar una regla que el laboratorio sí impone: **el número de rata es único dentro de su grupo**, no dentro de su tanda.

`ESPECIMEN.idGrupo` se repone como columna redundante, y se declara `UNIQUE (idGrupo, numeroRata)` . Motivo doble: la regla queda declarada en vez de vivir en código, y los reportes —que comparan grupos entre sí— dejan de tener que pasar por `TANDA` cada vez que preguntan «todas las ratas del grupo X».

Esto es una decisión de esquema físico, no toca el grafo relacional lógico.

`grafo_relacional_reconciliado.puml` se queda en BCNF estricta, sin `idGrupo` en `ESPECIMEN` — la redundancia se declara únicamente aquí, en el DDL, que es exactamente donde el método la permite (actividad 5 de la etapa física, no de la lógica).

```
CREATE TABLE ESPECIMEN
(
  idEspecimen      integer GENERATED ALWAYS AS IDENTITY,
  idTanda          integer NOT NULL,
  idGrupo          integer NOT NULL,  -- redundante a propósito · D-04
  numeroRata       smallint NOT NULL,
  numeroCilindro   smallint NOT NULL,

  CONSTRAINT PK_ESPECIMEN PRIMARY KEY (idEspecimen),
```

```

CONSTRAINT AK_ESPECIMEN_CILINDRO UNIQUE (idTanda, numeroCilindro),
CONSTRAINT AK_ESPECIMEN_RATA_GRUPO UNIQUE (idGrupo, numeroRata),

CONSTRAINT FK_TANDA_ESPECIMEN FOREIGN KEY (idTanda)
REFERENCES TANDA (idTanda) MATCH SIMPLE
ON UPDATE NO ACTION ON DELETE NO ACTION,
CONSTRAINT FK_GRUPO_ESPECIMEN FOREIGN KEY (idGrupo)
REFERENCES GRUPO (idGrupo) MATCH SIMPLE
ON UPDATE NO ACTION ON DELETE NO ACTION,

CONSTRAINT CK_ESPECIMEN_RATA CHECK (numeroRata >= 1)

-- falta: numeroCilindro <= nCilindros de su tanda, que mira otra tabla
-- idGrupo se mantiene sincronizado con un disparador, ver abajo
);

```

CÓMO SE MANTIENE IDGRUPO SINCRONIZADO CON LA TANDA

Guardar el mismo dato dos veces solo es seguro si algo garantiza que nunca se desincronizan. Un `CHECK` no basta porque compara con otra tabla; la solución es un disparador que recalcula `idGrupo` él solo cada vez que se inserta o cambia `idTanda`, en vez de confiar en que quien inserte la fila lo escriba bien.

```

CREATE FUNCTION fn_sincronizar_grupo_especimen()
RETURNS TRIGGER AS $$
BEGIN
    SELECT idGrupo INTO NEW.idGrupo
    FROM TANDA
    WHERE idTanda = NEW.idTanda;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_sincronizar_grupo_especimen
BEFORE INSERT OR UPDATE OF idTanda ON ESPECIMEN
FOR EACH ROW
EXECUTE FUNCTION fn_sincronizar_grupo_especimen();

```

POR QUÉ ASÍ, Y NO DE OTRA FORMA

El disparador corre **antes** de guardar la fila (`BEFORE`), no después: en vez de guardar lo que sea que llegó y luego corregirlo, reemplaza el valor de `idGrupo` antes de que se escriba. Con esto, la aplicación ni siquiera necesita mandar `idGrupo` al insertar o mover una rata de tanda — la base de datos lo calcula sola a partir de `idTanda` , así que nunca hay una versión "vieja" que se quede pegada.

Se dispara solo con `INSERT` o con un `UPDATE` que cambie `idTanda` específicamente (`UPDATE OF idTanda`) — no se ejecuta de más si se actualiza cualquier otra columna de `ESPECIMEN` , como `numeroCilindro` .

Lo demás del rendimiento

Fuera de D-04, esta actividad no tenía mucho que resolver y conviene decirlo: con el orden de magnitud calculado en la actividad 3 —cientos de miles de filas cortas en el peor caso—, **no hay un problema de rendimiento que atacar**. Lo que sí puede tardar es el procesamiento de video del pipeline, que no toca la base de datos y no se resuelve aquí.

ACTIVIDAD 6 RESUELTA

Documentación de la implementación

«Documentar las consideraciones tenidas en cuenta para la implementación de la base de datos física.»

METODOLOGÍA DE DISEÑO · ETAPA FÍSICA · ACTIVIDAD 6

Esta actividad es este documento, y solo podía cerrarse cuando las cinco anteriores lo hicieran — ya lo hicieron. Lo que deja fijado es la **trazabilidad**: cada elección de arriba tiene una fuente, y las fuentes son de dos clases muy distintas.

DECISIÓN	FUENTE	FECHA	ESTADO
PostgreSQL como gestor	D-11	13-sep	Decidida por el equipo
Administrador como subtipo	D-08	13-sep	Decidida y aplicada
USUARIO.activo en vez de borrar	D-10	13-sep	Decidida y aplicada

DECISIÓN	FUENTE	FECHA	ESTADO
Identificador institucional	D-02	12-sep	Decidida y aplicada
Tipos de dato	Este doc.	13-sep	Propuesta
Los ocho índices de foránea	Este doc.	13-sep	Propuesta
Nombrar todas las restricciones	Este doc.	13-sep	Propuesta
Unidad de <code>numeroRata</code> : redundancia controlada	D-04	13-sep	Decidida por el equipo
Vista de Administrador: se esconde y se bloquea, control en la aplicación	D-12	13-sep	Decidida por el equipo
Investigadores ven todos los experimentos, para comparar resultados	D-13	13-sep	Decidida por el equipo
<code>MODELO.hashModelo</code> se calcula con SHA-256, <code>char(64)</code>	D-14	13-sep	Decidida por el equipo
<code>ANALISIS.estado</code> son 4 valores, no 3 — se agrega en cola	D-15	13-sep	Decidida por el equipo
<code>ANALISIS.etapa</code> : <code>preprocesamiento</code> , <code>deteccion</code> , <code>seguimiento</code> , <code>clasificacion</code>	D-16	13-sep	Decidida por el equipo
<code>ANALISIS.nivelClasif</code> : <code>preciso</code> , <code>agrupado</code>	D-17	13-sep	Decidida por el equipo
Disparador de sincronización <code>ESPECIMEN.idGrupo</code>	D-04	13-sep	Escrito

DECISIÓN	FUENTE	FECHA	ESTADO
Respaldo diario con <code>pg_dump</code> , retención de 7 días	D-18	13-sep	Decidida por el equipo

CIERRE

Las decisiones están cerradas; la ejecución no

#	ACTIVIDAD	ESTADO	QUÉ FALTA EJECUTAR
01	Selección del gestor	Resuelta	—
02	Representación en el gestor	Resuelta	Escribir el DDL completo de las 16 relaciones
03	Archivos e índices	Resuelta	Declarar los 8 índices y configurar el cron del respaldo (D-18)
04	Seguridad y control de acceso	Resuelta	Escribir la comprobación de rol (D-12) y decidir cómo se identifica una sesión activa
05	Rendimiento y redundancia	Resuelta	Correr el disparador de sincronización de <code>idGrupo</code> ya escrito
06	Documentación	Resuelta	—

QUÉ SIGNIFICA "RESUELTA" AQUÍ

Las dos etapas anteriores se cerraron con resultado validado: un esquema conceptual y uno lógico que ya existen. Esta etapa cierra distinto, y conviene decirlo así en la defensa en vez de dejarlo implícito: **las seis actividades tienen su decisión tomada** —con fecha, con motivo, con equipo responsable, registradas en `preguntas-doctor.md` — pero el esquema físico todavía no se ha ejecutado sobre un PostgreSQL real. No hay tablas creadas, no hay disparador corriendo, no hay respaldo programado.

Eso no es un hueco del método: el diseño físico decide *qué* hacer, la implementación es *hacerlo*. Lo que sí vale la pena señalar es que hace apenas unas horas esta misma etapa tenía tres decisiones bloqueando todo el avance —la contradicción de permisos, la

unicidad de `numeroRata` , y ocho dominios sin valores— y ninguna de las tres sigue abierta.

TT 2026-B066 · ESCOM-IPN Frausto Robles Á. A. · Rodríguez Verdín S. V.

Etapas de diseño físico · actividades 1 a 6 · decisiones cerradas Actualizado 13-09-2026